

Section 14 — JavaScript Built-in Utilities

JavaScript শেখার সময় কিছু built-in utility বারবার ব্যবহার করতে হয়—বিশেষ করে **debugging, data conversion, timers, dates** এবং **code documentation**-এর জন্য।

এই section-এ আমরা শিখব:

1. `console.log()`
2. `console.error()`
3. `console.warn()`
4. `console.table()`
5. `console.dir()`
6. `console.group()`
7. `console.time()`
8. `console.timeEnd()`
9. `JSON.stringify()`
10. `JSON.parse()`
11. JSON-এর structure
12. JavaScript Object ↔ JSON conversion
13. `setTimeout()`
14. `setInterval()`
15. `clearTimeout()`
16. `clearInterval()`
17. Timer IDs
18. Timer cleanup
19. `Date` object basics
20. Current date/time
21. Date components
22. Date creation
23. Date getters
24. Date setters
25. Timestamp
26. Date comparison
27. Date formatting basics
28. Single-line comments `//`
29. Multi-line comments `/* */`
30. Comments-এর best practices
31. Common mistakes
32. Real-world examples
33. Practice problems
34. Interview questions
35. Quick revision
36. Final mental model

1. Built-in Utilities কী?

JavaScript-এর কিছু built-in functionality আছে যেগুলো everyday development-এ আমাদের কাজ সহজ করে।

যেমন:

```
Debugging
↓
console.*

Data conversion
↓
JSON.*

Timers
↓
setTimeout / setInterval

Date & Time
↓
Date

Documentation
↓
Comments
```

এগুলো কোনো external library ছাড়াই JavaScript environment/browser থেকে পাওয়া যায়।

2. console.log()

সবচেয়ে common debugging utility:

```
console.log("Hello World");
```

Output:

```
Hello World
```

Variable check করতে:

```
const name = "Shariar";  
  
console.log(name);
```

Output:

```
Shariar
```

3. Multiple Values

একসাথে multiple value log করা যায়:

```
const name = "Shariar";  
const age = 31;  
  
console.log(name, age);
```

Output:

```
Shariar 31
```

4. Label সহ Log

Debugging-এর সময় label দিলে অনেক clear হয়:

```
const price = 500;  
  
console.log("Price:", price);
```

আরও useful:

```
console.log("User:", user);
```

5. Object Log করা

```
const user = {  
  name: "Sharir",  
  age: 31  
};  
  
console.log(user);
```

Browser console-এ object expand করে properties দেখা যায়।

6. console.error()

Error বা failure-related information দেখাতে:

```
console.error("Something went wrong");
```

এটি সাধারণত browser console-এ error হিসেবে আলাদাভাবে দেখানো হয়।

Example:

```
if (!user) {  
  console.error("User not found");  
}
```

7. console.warn()

Warning দেখাতে:

```
console.warn("This feature is deprecated");
```

এটি error নয়, বরং developer-কে সতর্ক করার জন্য।

Mental model:

```
console.log()  
→ Information  
  
console.warn()  
→ Warning  
  
console.error()  
→ Error
```

8. console.table()

Array বা object-এর structured data table আকারে দেখাতে:

```
const users = [  
  {  
    name: "Sharir",  
    age: 31  
  },  
  {  
    name: "Rahim",  
    age: 28  
  }  
];  
  
console.table(users);
```

Console-এ data table-এর মতো দেখা যাবে।

এটি multiple records inspect করার জন্য খুব useful।

9. console.table() Object-এর সাথে

```
const user = {  
  name: "Sharir",  
  age: 31,  
  role: "Frontend Developer"  
};  
  
console.table(user);
```

Object-এর properties/value table-like format-এ দেখা যাবে।

10. কখন `console.table()` ব্যবহার করবে?

যখন data এমন:

Array of objects

যেমন:

```
const products = [
  { name: "Laptop", price: 800 },
  { name: "Mouse", price: 30 },
  { name: "Keyboard", price: 50 }
];
```

তখন:

```
console.table(products);
```

সাধারণ `console.log()` -এর চেয়ে অনেক easier to inspect হতে পারে।

11. `console.dir()`

`console.dir()` object-এর properties দেখতে useful।

```
const user = {
  name: "Sharar",
  age: 31
};

console.dir(user);
```

বিশেষ করে DOM element inspect করার সময় এটি useful হতে পারে:

```
const button = document.querySelector("button");  
  
console.dir(button);
```

এতে element-এর JavaScript object representation এবং properties/methods দেখতে সুবিধা হয়।

12. console.log() vs console.dir()

দুটাই debugging-এর জন্য useful।

```
console.log()  
→ সাধারণ output / object display  
  
console.dir()  
→ object-এর properties/methods inspect করার জন্য useful
```

DOM-এর ক্ষেত্রে difference আরও noticeable হতে পারে।

13. console.group()

Related logs-গুলো group করার জন্য:

```
console.group("User Info");  
  
console.log("Name:", "Shariar");  
console.log("Age:", 31);  
console.log("Role:", "Developer");  
  
console.groupEnd();
```

Console-এ একটি collapsible group পাওয়া যাবে।

14. Nested Console Groups

Group-এর ভিতর group করা যায়:

```
console.group("User");  
  
console.log("Name: Sharar");  
  
console.group("Address");  
  
console.log("City: Rangpur");  
  
console.groupEnd();  
  
console.groupEnd();
```

Complex debugging-এর সময় useful।

15. console.groupCollapsed()

Group default অবস্থায় collapsed রাখতে:

```
console.groupCollapsed("Debug Info");  
  
console.log("Step 1");  
console.log("Step 2");  
  
console.groupEnd();
```

16. console.time()

কোনো operation কত সময় নিচ্ছে measure করতে:

```
console.time("loop");  
  
for (let i = 0; i < 1000000; i++) {  
  // work  
}  
  
console.timeEnd("loop");
```

Console-এ roughly:


```
loop: X ms
```

দেখাবে।

17. `console.timeEnd()`

`console.time()` দিয়ে timer শুরু করতে হয়:

```
console.time("test");
```

তারপর:

```
console.timeEnd("test");
```

একই label ব্যবহার করতে হবে।

```
console.time("operation");  
  
// operation  
  
console.timeEnd("operation");
```

18. `console.time()` কেন Useful?

Performance comparison-এর basic debugging-এ:

```
console.time("method A");  
  
methodA();  
  
console.timeEnd("method A");
```

তারপর:

```
console.time("method B");
```

```
methodB();  
  
console.timeEnd("method B");
```

দুইটা operation কত সময় নিয়েছে compare করা যায়।

এটি production-grade benchmarking-এর replacement নয়। Precise performance analysis-এর জন্য dedicated profiling/benchmarking tools বেশি appropriate।

19. Console Utilities Quick Table

Method	Main Purpose
<code>console.log()</code>	সাধারণ debugging
<code>console.error()</code>	Error information
<code>console.warn()</code>	Warning
<code>console.table()</code>	Tabular data
<code>console.dir()</code>	Object inspection
<code>console.group()</code>	Logs group করা
<code>console.groupEnd()</code>	Group শেষ করা
<code>console.time()</code>	Timer শুরু
<code>console.timeEnd()</code>	Timer শেষ

20. JSON কী?

JSON = JavaScript Object Notation

JSON একটি lightweight **data interchange format**।

API-এর মাধ্যমে frontend/backend-এর মধ্যে data exchange করার সময় JSON খুব common।

Example:

```
{  
  "name": "Sharlar",  
  "age": 31,  
}
```

```
    "role": "Developer"
  }
```

দেখতে JavaScript object-এর মতো হলেও JSON এবং JavaScript object এক জিনিস নয়।

21. JavaScript Object

```
const user = {
  name: "Shariar",
  age: 31
};
```

এটি JavaScript object।

22. JSON String

```
const jsonUser = '{"name":"Shariar","age":31}';
```

এটি একটি **string**।

Check:

```
console.log(typeof jsonUser);
```

Output:

```
string
```

23. JSON.stringify()

JavaScript value/object-কে JSON string-এ convert করতে:

```
const user = {
  name: "Shariar",
  age: 31
};
```

```
};  
  
const jsonUser = JSON.stringify(user);  
  
console.log(jsonUser);
```

Output:

```
{"name": "Shariar", "age": 31}
```

Mental model:

```
JavaScript Object  
  ↓  
JSON.stringify()  
  ↓  
JSON String
```

24. JSON.parse()

JSON string-কে JavaScript value/object-এ convert করতে:

```
const jsonUser = '{"name": "Shariar", "age": 31}';  
  
const user = JSON.parse(jsonUser);  
  
console.log(user);
```

Mental model:

```
JSON String  
  ↓  
JSON.parse()  
  ↓  
JavaScript Object
```

25. stringify() vs parse()

সবচেয়ে important table:

Method	Conversion
<code>JSON.stringify()</code>	JS value → JSON string
<code>JSON.parse()</code>	JSON string → JS value

সহজভাবে:

```
stringify
→ String বানায়

parse
→ String গড়তে/convert করতে
```

26. API Response এবং JSON

ধরো backend response:

```
{
  "name": "Sharlar",
  "age": 31
}
```

JavaScript-এ API libraries/runtime সাধারণত response data নিয়ে parsing-এর কাজ সহজ করে দেয়।

Modern `fetch()` -এ:

```
const response = await fetch("/api/user");

const data = await response.json();
```

এখানে `.json()` response body থেকে JSON data parse করে JavaScript value দেয়।

তাই প্রতিটি `fetch()` response-এর জন্য manually:

```
JSON.parse()
```

করতে হয় না।

27. `JSON.stringify()` দিয়ে **API Request Body**

Backend-এ JSON পাঠানোর সময়:

```
const user = {  
  name: "Sharar",  
  age: 31  
};  
  
fetch("/api/users", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify(user)  
});
```

এখানে:

```
JavaScript object  
  ↓  
JSON.stringify()  
  ↓  
JSON request body  
  ↓  
Backend
```

28. JSON-এর **Valid Data Types**

JSON-এ সাধারণত support করে:

```
String  
Number  
Boolean
```

```
null
Object
Array
```

Example:

```
{
  "name": "Shariar",
  "age": 31,
  "isDeveloper": true,
  "skills": ["JavaScript", "React"],
  "address": null
}
```

29. JSON-এ JavaScript-এর সব কিছু রাখা যায় না

JSON supports না:

```
undefined
function
Symbol
BigInt
```

যেমন:

```
const data = {
  name: "Shariar",
  greet() {
    console.log("Hello");
  }
};
```

`JSON.stringify()` করার সময় function property JSON-এর মধ্যে থাকবে না।

30. JSON Stringification Example

```
const data = {
  name: "Shariar",
```

```
    age: 31,  
    active: true  
};  
  
const json = JSON.stringify(data);  
  
console.log(json);
```

Result:

```
{"name":"Shariar","age":31,"active":true}
```

31. JSON Parse Error

Invalid JSON parse করলে error হতে পারে।

```
const data = '{"name":"Shariar"}';  
  
JSON.parse(data);
```

এখানে JSON malformed হওয়ায় `SyntaxError` হতে পারে।

Safe handling:

```
try {  
    const data = JSON.parse(jsonString);  
  
    console.log(data);  
} catch (error) {  
    console.error("Invalid JSON:", error);  
}
```

এটি আগের **Error Handling** section-এর সাথে directly related।

32. JSON Deep Copy Pattern

একটি common old technique:


```
const copy = JSON.parse(JSON.stringify(original));
```

এটি কিছু simple data structure-এর deep copy করতে পারে।

কিন্তু এটি universal deep clone solution নয়।

কারণ JSON conversion-এ কিছু JavaScript values/type information হারাতে পারে।

যেমন:

```
Date  
Map  
Set  
undefined  
Function  
Symbol  
BigInt
```

তাই modern JavaScript-এ supported data হলে:

```
structuredClone()
```

অনেক ক্ষেত্রে better choice।

33. Timer Utilities

JavaScript-এ delayed বা repeated কাজ schedule করতে timers ব্যবহার করা যায়।

Main four:

```
setTimeout()  
setInterval()  
clearTimeout()  
clearInterval()
```

34. `setTimeout()`

নির্দিষ্ট delay-এর পরে একবার callback execute করে।

```
setTimeout(() => {  
  console.log("Hello");  
}, 2000);
```

এখানে callback প্রায় 2 seconds পরে run করার জন্য schedule হয়েছে।

Important:

`setTimeout()` **exact execution time guarantee** করে না।

এটি minimum delay-এর পর callback queue করার জন্য schedule করে; event loop এবং main thread busy থাকলে আরও পরে execute হতে পারে।

35. `setTimeout()` Syntax

```
setTimeout(callback, delay);
```

Example:

```
setTimeout(() => {  
  console.log("Done");  
}, 1000);
```

Flow:

```
setTimeout()  
  ↓  
Wait approximately 1 second  
  ↓  
Callback scheduled  
  ↓  
Event loop সুযোগ পেলে callback execute
```

36. `setTimeout()` with Arguments

```
function greet(name) {  
  console.log(`Hello ${name}`);  
}  
  
setTimeout(greet, 1000, "Shariar");
```

37. `setTimeout()` Timer ID

`setTimeout()` একটি timer ID/handle return করে।

```
const timerId = setTimeout(() => {  
  console.log("Hello");  
}, 5000);
```

এই ID পরে timer cancel করতে ব্যবহার করা যায়।

38. `clearTimeout()`

Scheduled timeout cancel করতে:

```
const timerId = setTimeout(() => {  
  console.log("Hello");  
}, 5000);  
  
clearTimeout(timerId);
```

এখন callback execute হওয়ার আগেই timeout cancel হয়ে যাবে।

Mental model:

```
setTimeout()  
  ↓  
timerId  
  ↓  
clearTimeout(timerId)
```

↓
cancel

39. setInterval()

নির্দিষ্ট interval পরপর callback repeatedly schedule করে।

```
const intervalId = setInterval(() => {  
  console.log("Running...");  
}, 1000);
```

প্রায় প্রতি 1 second পর callback চালানোর চেষ্টা করবে।

40. clearInterval()

Interval stop করতে:

```
const intervalId = setInterval(() => {  
  console.log("Running...");  
}, 1000);  
  
clearInterval(intervalId);
```

41. setTimeout() vs setInterval()

Method	Behavior
setTimeout()	একবার execute করার জন্য schedule
setInterval()	বারবার execute করার জন্য schedule
clearTimeout()	timeout cancel
clearInterval()	interval stop

Mental model:

```
setTimeout  
→ once  
  
setInterval  
→ repeat
```

42. Countdown Example

```
let count = 5;  
  
const intervalId = setInterval(() => {  
  console.log(count);  
  
  count--;  
  
  if (count < 0) {  
    clearInterval(intervalId);  
  }  
}, 1000);
```

Flow:

```
5  
4  
3  
2  
1  
0  
stop
```

43. Better Countdown Pattern

যদি **5** থেকে **1** পর্যন্ত দেখিয়ে stop করতে চাও:

```
let count = 5;  
  
const intervalId = setInterval(() => {  
  console.log(count);
```

```
count--;  
  
if (count === 0) {  
  clearInterval(intervalId);  
}  
}, 1000);
```

44. Timer Cleanup কেন Important?

Long-running interval বন্ধ না করলে unnecessary work চলতে পারে।

যেমন:

```
setInterval(() => {  
  fetchData();  
}, 1000);
```

এটি indefinitely চলবে যতক্ষণ না stop করা হয়।

তাই lifecycle শেষ হলে:

```
clearInterval(intervalId);
```

করতে হবে।

React-এ এই concept `useEffect` cleanup-এর সাথে খুব closely related।

45. `setTimeout()` Recursive Pattern

কখনো repeated কাজের জন্য `setInterval()`-এর বদলে recursive `setTimeout()` ব্যবহার করা useful।

```
function run() {  
  console.log("Running");  
  
  setTimeout(run, 1000);  
}
```

```
run();
```

এখানে প্রতিবার callback নিজেই পরের timeout schedule করছে।

এটি async operations-এর timing control করার ক্ষেত্রে useful হতে পারে।

46. setInterval() vs Recursive setTimeout()

Conceptually:

```
setInterval  
→ fixed repeating schedule  
  
recursive setTimeout  
→ previous work শেষ হওয়ার পর next schedule করা সম্ভব
```

বিশেষ করে network polling-এর ক্ষেত্রে recursive timeout useful হতে পারে:

```
async function poll() {  
  await fetchData();  
  
  setTimeout(poll, 2000);  
}  
  
poll();
```

এতে আগের async operation শেষ হওয়ার পরে next polling schedule করা যায়।

47. Important Timer Concept

এই code:

```
setTimeout(() => {  
  console.log("Hello");  
}, 0);  
  
console.log("World");
```

Output:

```
World  
Hello
```

কারণ `setTimeout(..., 0)` callback immediate synchronous execution করে না।

এটি event loop-এর asynchronous scheduling-এর সাথে সম্পর্কিত।

এটি পরের **Asynchronous JavaScript / Event Loop** section-এ আরও গভীরভাবে আসবে।

48. Date Object

JavaScript-এর built-in `Date` object date এবং time নিয়ে কাজ করতে ব্যবহৃত হয়।

Current date/time:

```
const now = new Date();  
  
console.log(now);
```

49. `new Date()`

```
const now = new Date();
```

এটি current date/time represent করে।

50. Date String

Specific date create করা যায়:

```
const date = new Date("2026-09-02");  
  
console.log(date);
```


Date parsing-এর জন্য standardized ISO-style strings prefer করা ভালো:

```
YYYY-MM-DD
```

আর date-time-এর ক্ষেত্রে timezone explicitly consider করা important।

51. Date Components

Date object থেকে বিভিন্ন component পাওয়া যায়।

```
const date = new Date();

console.log(date.getFullYear());
console.log(date.getMonth());
console.log(date.getDate());
console.log(date.getDay());
```

52. `getFullYear()`

Year:

```
date.getFullYear();
```

Example:

```
2026
```

53. `getMonth()`

Month:

```
date.getMonth();
```

খুব important:

Month zero-based |

অর্থাৎ:

```
January → 0  
February → 1  
March → 2  
...  
December → 11
```

এটি beginner-এর খুব common mistake |

54. getDate()

Month-এর কততম দিন:

```
date.getDate();
```

Output:

```
1 → 31
```

55. getDay()

Week-এর day:

```
date.getDay();
```

এটিও zero-based:

```
Sunday → 0  
Monday → 1  
Tuesday → 2  
Wednesday → 3  
Thursday → 4
```

Friday → 5
Saturday → 6

56. Date Getters Quick Table

Method	Meaning
<code>getFullYear()</code>	Year
<code>getMonth()</code>	Month <code>0-11</code>
<code>getDate()</code>	Day of month
<code>getDay()</code>	Day of week <code>0-6</code>
<code>getHours()</code>	Hour
<code>getMinutes()</code>	Minutes
<code>getSeconds()</code>	Seconds
<code>getMilliseconds()</code>	Milliseconds

57. Time Getters

```
const now = new Date();  
  
console.log(now.getHours());  
console.log(now.getMinutes());  
console.log(now.getSeconds());
```

58. UTC Getters

Local time-এর পাশাপাশি UTC values-এর জন্য:

```
date.getUTCFullYear();  
date.getUTCMonth();  
date.getUTCDate();  
date.getUTCHours();
```

Timezone-sensitive application-এ local time এবং UTC-এর difference বুঝতে হবে।

59. Date Setters

Date-এর component পরিবর্তন করা যায়:

```
const date = new Date();  
  
date.setFullYear(2030);  
  
console.log(date);
```

আরও:

```
date.setMonth(5);  
date.setDate(15);  
date.setHours(10);
```

60. Date Timestamp

`Date` internally একটি numeric timestamp-এর সাথে কাজ করে।

```
const now = new Date();  
  
console.log(now.getTime());
```

এটি সাধারণত **milliseconds since Unix epoch** return করে।

Unix epoch:

```
1970-01-01T00:00:00.000Z
```

61. `Date.now()`

Current timestamp পাওয়ার সহজ উপায়:

```
const timestamp = Date.now();  
  
console.log(timestamp);
```

এটি milliseconds-এ current Unix timestamp দেয়।

62. Date Comparison

Date compare করার সময় timestamp ব্যবহার করা সহজ:

```
const first = new Date("2026-09-01");  
const second = new Date("2026-09-02");  
  
console.log(first < second);
```

Output:

```
true
```

কারণ Date objects numeric time value-এর ভিত্তিতে compare করা যায়।

63. Date Difference

দুই date-এর difference:

```
const start = new Date("2026-09-01");  
const end = new Date("2026-09-05");  
  
const difference = end - start;  
  
console.log(difference);
```

Result milliseconds-এ।

Days বের করতে:

```
const days = difference / (1000 * 60 * 60 * 24);  
  
console.log(days);
```

64. Date Formatting

Simple human-readable format:

```
const date = new Date();  
  
console.log(date.toString());
```

ISO format:

```
console.log(date.toISOString());
```

Example:

```
2026-09-02T...
```

API/backend data exchange-এর ক্ষেত্রে ISO 8601 format খুব common ।

65. toLocaleString()

User-এর locale অনুযায়ী readable date/time:

```
const date = new Date();  
  
console.log(date.toLocaleString());
```

Date only:

```
console.log(date.toLocaleDateString());
```

Time only:

```
console.log(date.toLocaleTimeString());
```

66. Better Date Formatting

Specific formatting options দেওয়া যায়:

```
const date = new Date();

console.log(
  date.toLocaleDateString("en-US", {
    year: "numeric",
    month: "long",
    day: "numeric"
  })
);
```

এতে readable format পাওয়া যায়।

67. Date-এর Important Warning

JavaScript-এর `Date` object-এর timezone behavior beginner-এর কাছে confusing হতে পারে।

বিশেষ করে:

```
new Date("2026-09-02")
```

এর interpretation এবং displayed local date timezone-এর কারণে unexpected হতে পারে।

তাই real-world application-এ:

- UTC vs local timezone বুঝতে হবে
- API-তে ISO timestamps prefer করা ভালো
- user-facing formatting-এর জন্য locale-aware formatting ব্যবহার করা ভালো
- complex timezone/calendar requirements-এর ক্ষেত্রে dedicated date/time library বা modern Temporal API যেখানে available/appropriate, consider করা যায়

68. Comments কী?

Code-এর মধ্যে developer-এর জন্য explanation বা note লিখতে comments ব্যবহার করা হয়।

Comments JavaScript engine execute করে না।

দুই ধরনের basic comment:

```
// Single-line comment
```

এবং:

```
/*  
    Multi-line comment  
*/
```

69. Single-Line Comment

```
// Calculate total price  
  
const total = price * quantity;
```

// থেকে line-এর শেষ পর্যন্ত comment।

70. Inline Comment

```
const total = price * quantity; // total price
```

Possible, কিন্তু excessive inline comments readability কমাতে পারে।

71. Multi-Line Comment

```
/*  
    This function calculates
```



```
    the total price of products.  
*/  
function calculateTotal(price, quantity) {  
    return price * quantity;  
}
```

Multiple lines explain করার জন্য useful ।

72. Comments-এর Best Practice

ভালো comment সাধারণত কেন কিছু করা হয়েছে সেটা explain করে ।

কম useful:

```
// Add 1 to count  
count++;
```

এখানে code নিজেই obvious ।

Better:

```
// Keep one extra slot for the pending item.  
count++;
```

অর্থাৎ:

```
Comment should explain WHY  
when the WHY is not obvious.
```

73. Outdated Comments

Code পরিবর্তন করে comment update না করলে সমস্যা হয় ।

খারাপ:

```
// Maximum 10 users allowed  
const maxUsers = 20;
```

Comment এবং code contradict করছে।

তাই stale comments dangerous।

74. Comment দিয়ে Code Hide করা

Development-এর সময় temporary debugging-এর জন্য:

```
// console.log(user);
```

ঠিক আছে।

কিন্তু permanently unused code comment করে রেখে দেওয়া উচিত নয়।

Unused code remove করা better।

75. Comments vs Documentation

Comment:

```
// Prevent duplicate requests
```

Documentation সাধারণত larger explanation/API usage দেয়।

তোমার codebase বড় হলে:

```
Comments  
→ local explanation  
  
Documentation  
→ broader usage/instructions
```

দুটো আলাদা purpose serve করে।

76. Real-World Debugging Example

ধরো API থেকে users এসেছে:

```
const users = [
  { id: 1, name: "Sharlar", role: "developer" },
  { id: 2, name: "Rahim", role: "designer" }
];

console.group("Users Debug");

console.log("Total users:", users.length);

console.table(users);

console.groupEnd();
```

এভাবে console debugging অনেক organized হয়।

77. Performance Debugging Example

```
console.time("process-users");

const result = users.map((user) => {
  return user.name.toUpperCase();
});

console.timeEnd("process-users");
```

এখানে processing time-এর rough measurement পাওয়া যায়।

78. JSON + Local Storage Connection

Browser `localStorage` values string হিসেবে store করে।

তাই object store করতে:

```
const user = {
  name: "Sharlar",
```

```
    age: 31
  };

  localStorage.setItem(
    "user",
    JSON.stringify(user)
  );
```

পরে:

```
const storedUser = localStorage.getItem("user");

const userData = JSON.parse(storedUser);
```

Flow:

```
Object
  ↓
JSON.stringify()
  ↓
String
  ↓
localStorage
  ↓
String
  ↓
JSON.parse()
  ↓
Object
```

এটি খুব common real-world pattern ।

79. Timer + UI Example

ধরো button click করার পরে message 3 seconds পর hide করতে হবে:

```
const message = document.querySelector("#message");

message.classList.add("visible");

const timerId = setTimeout(() => {
```

```
message.classList.remove("visible");  
}, 3000);
```

এখানে:

```
Click  
↓  
Show message  
↓  
3 seconds  
↓  
Hide message
```

80. Timer Cancellation Example

ধরো user আবার action করলে আগের timeout cancel করতে হবে:

```
let timerId;  
  
function showMessage() {  
  clearTimeout(timerId);  
  
  timerId = setTimeout(() => {  
    console.log("Hide message");  
  }, 3000);  
}
```

এভাবে multiple pending timeout জমে যাওয়া avoid করা যায়।

81. Date + Countdown Example

ধরো একটি deadline আছে:

```
const deadline = new Date("2026-12-31T23:59:59");  
  
const now = new Date();  
  
const remaining = deadline - now;
```

```
console.log(remaining);
```

এখানে remaining time milliseconds-এ পাওয়া যাবে।

তারপর:

```
const days = Math.floor(
  remaining / (1000 * 60 * 60 * 24)
);
```

এভাবে countdown calculation করা যায়।

82. Common Mistakes

Mistake 1 — `JSON.stringify()` আর `JSON.parse()` উল্টো করা

মনে রাখো:

```
stringify
→ object → string

parse
→ string → object
```

Mistake 2 — Invalid JSON parse করা

```
JSON.parse("{name: 'Sharlar'}");
```

এটি valid JSON নয়।

Valid JSON:

```
{
  "name": "Sharlar"
}
```

Mistake 3 — `setTimeout()` immediate execution ভাবা

```
setTimeout(callback, 0);
```

মানে callback এখনই synchronousভাবে execute হবে না।

Mistake 4 — Interval clear না করা

```
setInterval(() => {  
  console.log("Running");  
}, 1000);
```

এটি continuously চলবে।

প্রয়োজনে ID save করো:

```
const id = setInterval(...);  
  
clearInterval(id);
```

Mistake 5 — Month zero-based ভুলে যাওয়া

```
date.getMonth();
```

January:

```
0
```

December:

```
11
```

Mistake 6 — `getDate()` আর `getDay()` confuse করা

```
getDate()  
→ month-এর day  
  
getDay()  
→ week-এর day
```

Mistake 7 — `console.log()`-কে error handling ভাবা

```
console.error("Failed");
```

এটি শুধু console-এ error information দেখায়।

এটি exception throw করে না।

Exception throw করতে:

```
throw new Error("Failed");
```

83. Important Difference

```
console.error("Something went wrong");
```

এবং:

```
throw new Error("Something went wrong");
```

এক জিনিস নয়।

```
console.error()
```

```
→ developer console-এ message
```


throw

→ JavaScript error/exception flow শুরু

এটি আগের **Error Handling** section-এর সাথে directly connected।

84. Built-in Utilities Quick Reference

Console

```
console.log();  
console.error();  
console.warn();  
console.table();  
console.dir();  
console.group();  
console.groupEnd();  
console.time();  
console.timeEnd();
```

JSON

```
JSON.stringify();  
JSON.parse();
```

Timers

```
setTimeout();  
setInterval();  
clearTimeout();  
clearInterval();
```

Date

```
new Date();  
Date.now();  
  
date.getFullYear();
```

```
date.getMonth();
date.getDate();
date.getDay();

date.getHours();
date.getMinutes();
date.getSeconds();

date.toISOString();
date.toLocaleString();
```

Comments

```
// Single line
```

```
/*
  Multiple lines
*/
```

85. Practice Questions

Practice 1 — Console

একটি user object তৈরি করো:

```
{
  name: "Sharir",
  age: 31,
  role: "Developer"
}
```

এবং:

- `console.log()`
- `console.table()`
- `console.dir()`

দিয়ে inspect করো।

Practice 2 — Console Group

একটি "User Debug" group তৈরি করে:

```
Name  
Age  
Role
```

log করো।

Practice 3 — Performance

একটি loop কত milliseconds নেয় সেটা:

```
console.time()  
console.timeEnd()
```

দিয়ে measure করো।

Practice 4 — JSON

এই object:

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

কে:

```
JSON string
```

এ convert করো।

তারপর আবার object-এ convert করো।

Practice 5 — JSON Error Handling

Invalid JSON parse করার চেষ্টা করো এবং `try/catch` দিয়ে error handle করো।

Practice 6 — Timeout

5 seconds পরে:

```
"Hello Shariar"
```

print করো।

Practice 7 — Cancel Timeout

একটি 5-second timeout তৈরি করো এবং 2 seconds-এর মধ্যে সেটি cancel করো।

Practice 8 — Interval

প্রতি 1 second পর:

```
Hello
```

print করো।

5 বার হওয়ার পর interval stop করো।

Practice 9 — Date

আজকের:

```
Year  
Month  
Date  
Day  
Hour
```

Minute
Second

console-এ দেখাও।

Practice 10 — Date Difference

দুটি date-এর মধ্যে কত দিন difference আছে বের করো।

86. Output Prediction

Question 1

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 0);  
  
console.log("C");
```

Output কী হবে?

Question 2

```
const date = new Date("2026-01-15");  
  
console.log(date.getMonth());  
console.log(date.getDate());
```

Output কী হবে?

Question 3

```
const user = {  
  name: "Sharar",  
  age: 31  
};
```

```
const json = JSON.stringify(user);  
  
console.log(typeof json);
```

Output কী হবে?

Question 4

```
const json = '{"name":"Shariar"}';  
  
const user = JSON.parse(json);  
  
console.log(typeof user);
```

Output কী হবে?

87. Coding Challenges

Challenge 1 — Auto Hide Message

একটি function তৈরি করো:

```
showMessage("Saved successfully");
```

যা:

1. Message show করবে
 2. 3 seconds পরে hide করবে
-

Challenge 2 — Countdown

একটি countdown তৈরি করো:

```
5  
4  
3  
2
```

1
Done

`setInterval()` ব্যবহার করে।

Challenge 3 — JSON Storage

একটি user object:

```
const user = {  
  name: "Sharlar",  
  role: "Developer"  
};
```

`localStorage` -এ save করে এবং পরে retrieve করে object হিসেবে ব্যবহার করে।

Challenge 4 — Date Formatter

একটি function তৈরি করে:

```
formatDate(new Date());
```

যেটা readable date return করবে।

88. Interview Questions

1. `console.log()` এবং `console.error()` -এর difference কী?

`console.log()` সাধারণ debugging information-এর জন্য, আর `console.error()` error-related information display করার জন্য।

2. `console.table()` কখন ব্যবহার করবে?

Array of objects বা structured data table আকারে inspect করতে।

3. `JSON.stringify()` কী করে?

JavaScript value-কে JSON string-এ convert করে।

4. `JSON.parse()` কী করে?

Valid JSON string-কে JavaScript value-এ convert করে।

5. `setTimeout()` কি **exact** সময় পরে **callback execute** করার **guarantee** দেয়?

না। এটি specified delay-এর আগে execute করবে না; event loop/main thread busy থাকলে আরও পরে execute হতে পারে।

6. `setTimeout()` এবং `setInterval()` এর **difference** কী?

```
setTimeout  
→ one scheduled execution  
  
setInterval  
→ repeated scheduled executions
```

7. `clearTimeout()` কেন দরকার?

Scheduled timeout cancel করতে।

8. `clearInterval()` কেন দরকার?

Running interval stop করতে।

9. `Date.getMonth()` কেন **confusing**?

কারণ month zero-based:

```
January = 0  
December = 11
```

10. `getDate()` এবং `getDay()` কী?

`getDate()`
→ month-এর day

`getDay()`
→ week-এর day

11. `Date.now()` কী **return** করে?

Current time-এর Unix timestamp milliseconds-এ।

12. **JSON** এবং **JavaScript object** কি একই?

না।

JSON হলো data interchange format; JavaScript object হলো JavaScript runtime-এর data structure।

13. `console.error()` কি **exception throw** করে?

না।

```
console.error("Failed");
```

শুধু console-এ error message দেখায়।

Exception throw করতে:

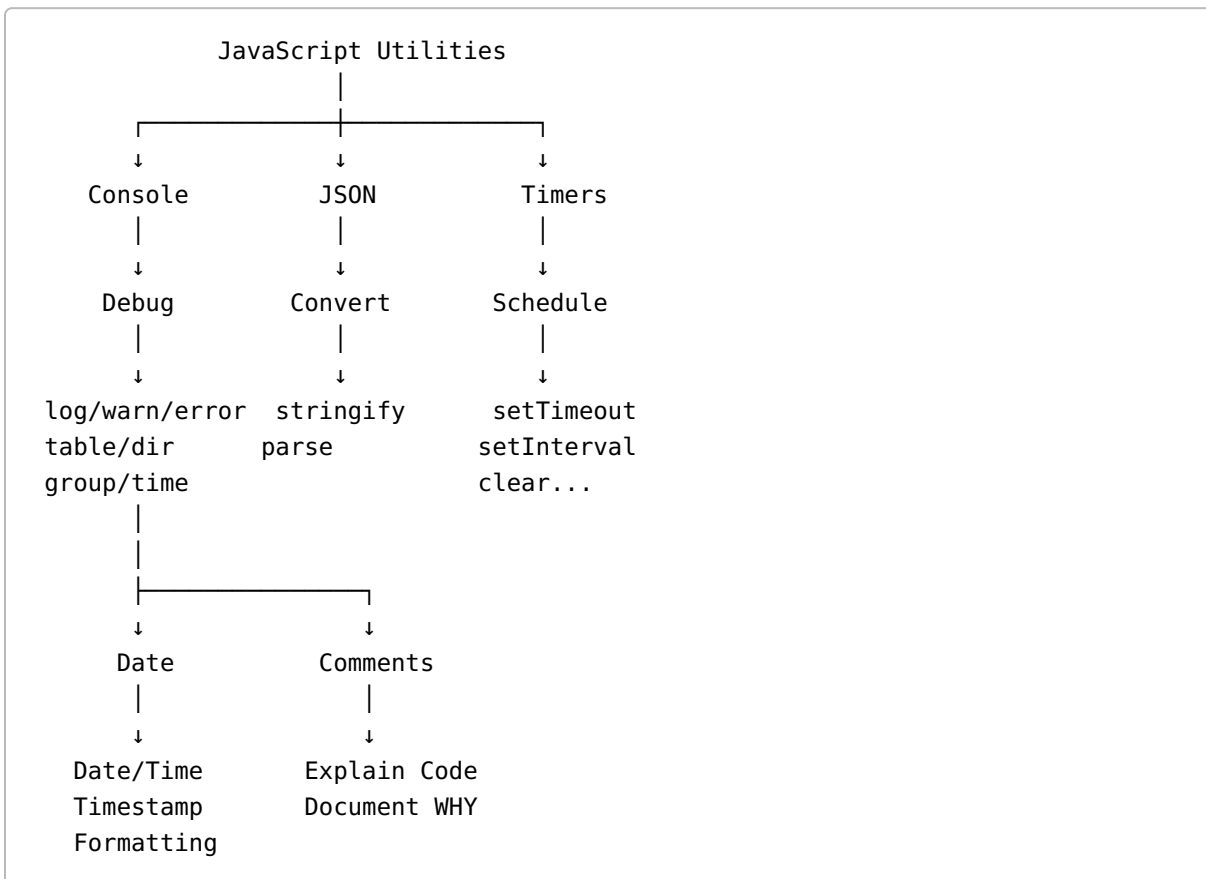
```
throw new Error("Failed");
```

14. **Event loop**-এর সাথে `setTimeout()`-এর **relation** কী?

Timer callback delay-এর পরে scheduling-এর জন্য eligible হয়; event loop এবং call stack available হলে callback execute হয়।

89. Final Mental Model

পুরো Built-in Utilities section:



সবচেয়ে Important Mental Shortcuts

console

→ Debug

JSON.stringify

→ JS → String

JSON.parse

→ String → JS

setTimeout

→ Later, once

```
setInterval
→ Repeatedly

clearTimeout
→ Cancel timeout

clearInterval
→ Stop interval

Date
→ Date + Time

//
→ One-line comment

/*
*/
→ Multi-line comment
```

✓ Section 14 Mastery Checklist

- [] `console.log()` বুঝি
- [] `console.error()` বুঝি
- [] `console.warn()` বুঝি
- [] `console.table()` ব্যবহার করতে পারি
- [] `console.dir()` বুঝি
- [] `console.group()` / `console.groupEnd()` বুঝি
- [] `console.time()` / `console.timeEnd()` বুঝি
- [] JSON কী বুঝি
- [] JavaScript object vs JSON string বুঝি
- [] `JSON.stringify()` পারি
- [] `JSON.parse()` পারি
- [] JSON parsing error handle করতে পারি
- [] `setTimeout()` পারি
- [] `setInterval()` পারি
- [] `clearTimeout()` পারি
- [] `clearInterval()` পারি
- [] Timer ID/handle বুঝি
- [] Timer cleanup বুঝি
- [] `setTimeout(..., 0)` synchronous নয় বুঝি
- [] `Date` object তৈরি করতে পারি
- [] `Date.now()` বুঝি
- [] `getFullYear()` বুঝি
- [] `getMonth()` -এর zero-based behavior জানি

- [] `getDate()` এবং `getDay()` difference জানি
- [] Time getters জানি
- [] UTC getters সম্পর্কে ধারণা আছে
- [] Date comparison করতে পারি
- [] Date difference বের করতে পারি
- [] `toISOString()` বুঝি
- [] `toLocaleString()` বুঝি
- [] `//` comments বুঝি
- [] `/* */` comments বুঝি
- [] ভালো comment কীভাবে লিখতে হয় জানি
- [] `console.error()` vs `throw new Error()` বুঝি
- [] JSON + localStorage connection বুঝি
- [] Timer cleanup-এর importance বুঝি

Final Golden Rules

1. **Debugging**-এর জন্য শুধু `console.log()` -এর উপর নির্ভর করো না। `table`, `dir`, `group`, `time` -ও ব্যবহার করো।
2. `JSON.stringify()` = JavaScript value → JSON string।
3. `JSON.parse()` = JSON string → JavaScript value।
4. `setTimeout()` একবার **schedule** করে; `setInterval()` **repeatedly schedule** করে।
5. **Timer**-এর ID save করলে পরে **timer cancel/stop** করা যায়।
6. `setTimeout(..., 0)` মানে **immediate execution** নয়।
7. `getMonth()` **zero-based—January** হলো `0`।
8. `getDate()` হলো **month-এর day**, `getDay()` হলো **week-এর day**।
9. **Date/time**-এর ক্ষেত্রে **UTC** এবং **local timezone**-এর **difference** বুঝে কাজ করো।
10. `console.error()` শুধু **console-এ error** দেখায়; **exception throw** করতে `throw new Error()` লাগে।
11. **Comments code**-এর **obvious** অংশ **explain** করার চেয়ে **non-obvious WHY explain** করতে বেশি **valuable**।
12. **JSON + localStorage** হলো **frontend-এর খুব common data persistence pattern**।



Built-in Utilities শেখার পর তোমার Foundation

এখন পর্যন্ত JavaScript-এর foundation অনেকটা এমন:

Variables & Data Types



Strings



Numbers & Math



Arrays



Objects



Control Flow



Loops



Functions



Scope & Hoisting



Error Handling



DOM



DOM Events



Built-in Utilities

এর পরের বড় jump হবে:

Synchronous JavaScript



Call Stack



Web APIs



Callback Queue



Event Loop



Promises



async / await

↓
Fetch API
↓
Asynchronous JavaScript

এখান থেকেই JavaScript-এর **real asynchronous behavior** গভীরভাবে বোঝা শুরু হবে।